



Scalable and Responsive with Event-Driven Design

ABOUT OUR BRAND GUIDELINES

Objectives:

Describe Event-Driven Architecture and its core components.

Recognize key EDA design patterns and use cases.

Weigh the benefits and trade-offs of EDA.

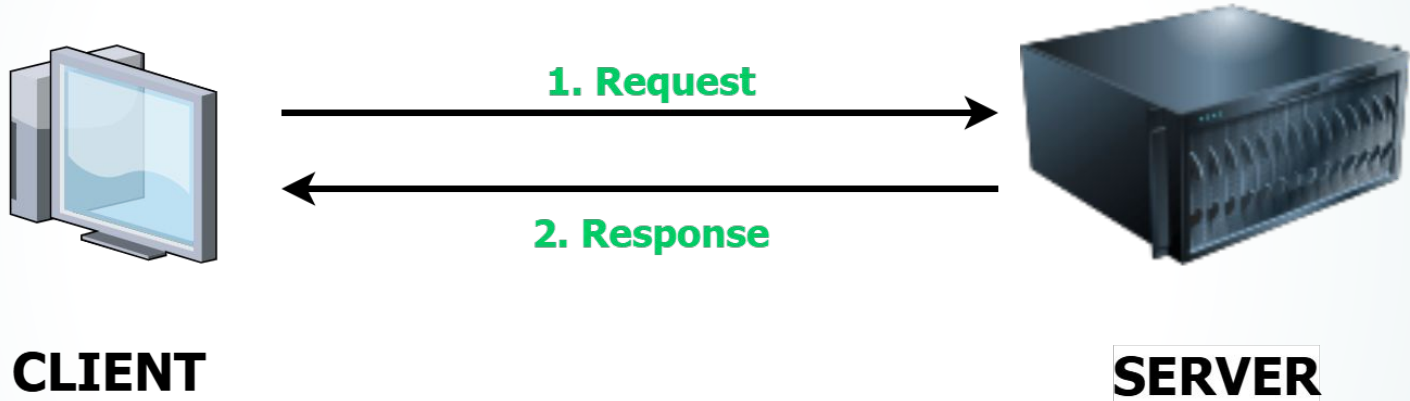
Build a simple event-driven microservice system

TABLE OF CONTENTS

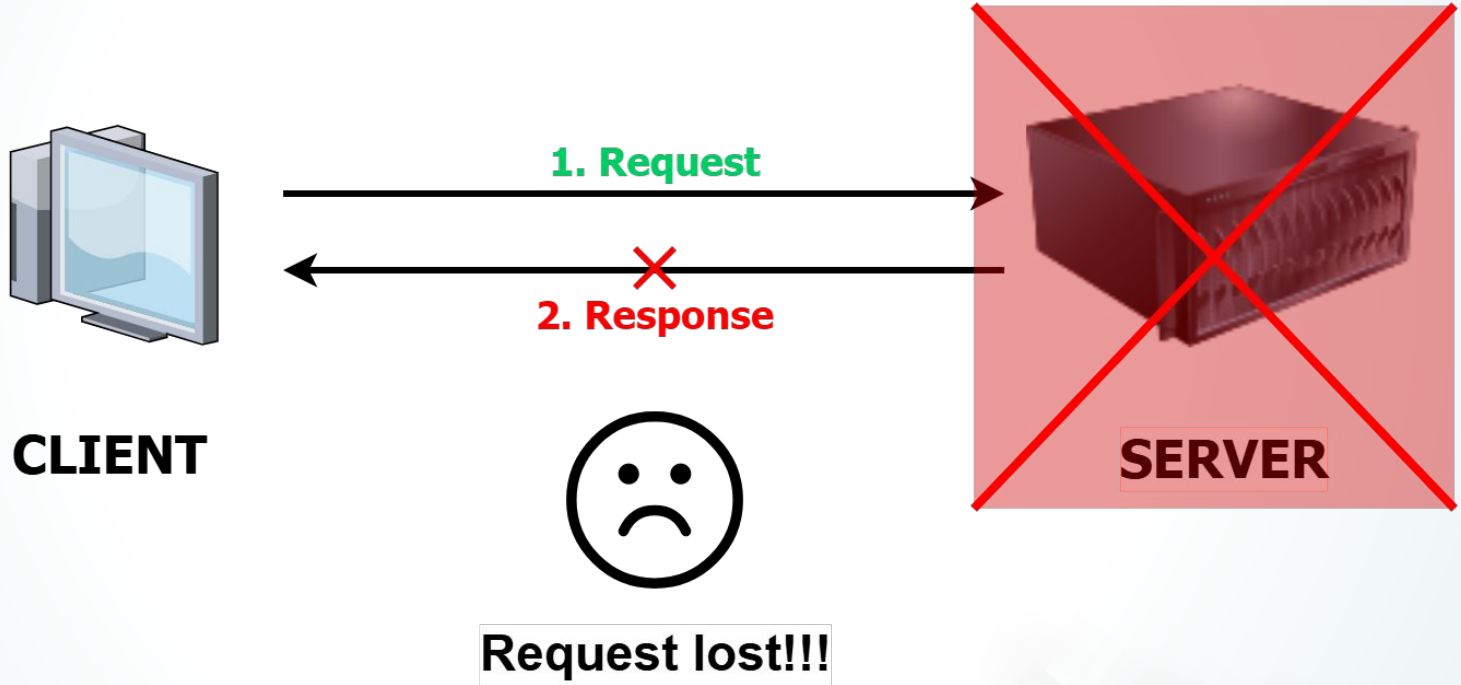
1. Introduction to Event-Driven Architecture (EDA)
2. Core concepts and Design patterns
3. Trade-offs
4. Demo: Building an event-driven system
5. Discussion and Q&A

Introduction to Event-Driven Architecture

Request/Response model



Request/Response model



Request/Response model (cont.)



What is Event-Driven Architecture (EDA)?

Event-Driven Architecture (EDA) is a software design paradigm in which the flow of the program is determined by events - changes in state or occurrences of business moments that are meaningful to the system. Unlike traditional request-response architectures where components directly call each other and wait for responses, EDA enables a reactive system where components produce and consume events asynchronously.



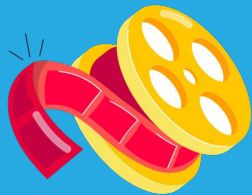
Promotes loose coupling and asynchronous communication.



Events are produced, routed, and consumed - often using a message broker.

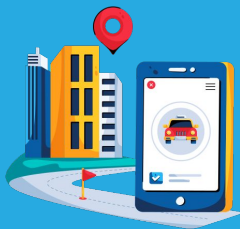


Common in systems that need scalability, responsiveness, and resilience.



Film reel

- ❑ Publishes events when a user plays, pauses, or finishes content.
- ❑ Services (recommendations, analytics, billing) process events asynchronously.



Booking

- ❑ Continuously emits driver and rider location events.
- ❑ Matching, pricing, and ETA services subscribe and respond in near real-time..



IoT Systems

- ❑ Devices send events (temperature, motion, humidity) to a central broker.
- ❑ Consumers trigger alerts, automation, or data storage asynchronously.



Banking (Fraud Detection)

- ❑ Transactions publish events to fraud detection systems.
- ❑ Machine learning models consume events to flag suspicious activity.

Core concepts and Design patterns

Events and Event Types

Event 🟢:

A significant change in state that is meaningful to the system.

Example: "OrderPlaced", "PaymentProcessed".

Event Message 🟡:

A message that carries details about an event.

Payload: The actual data (e.g., order ID, customer info).

Metadata: Context about the event (e.g., timestamp, event type, source).



Events and Event Types

 Domain Event	 Integration Event
Business-meaningful occurrences that domain experts care about	Events designed for cross-service communication
Example: "OrderShipped" in the Order service.	Example: "OrderCreated" sent from Order service to Inventory service.
Usually not designed for other domains to consume.	Explicitly intended for other domains or systems.

Key Components



PRODUCER

Generates and publishes events.
Example: Order service sending
“OrderPlaced” event.



BROCKER

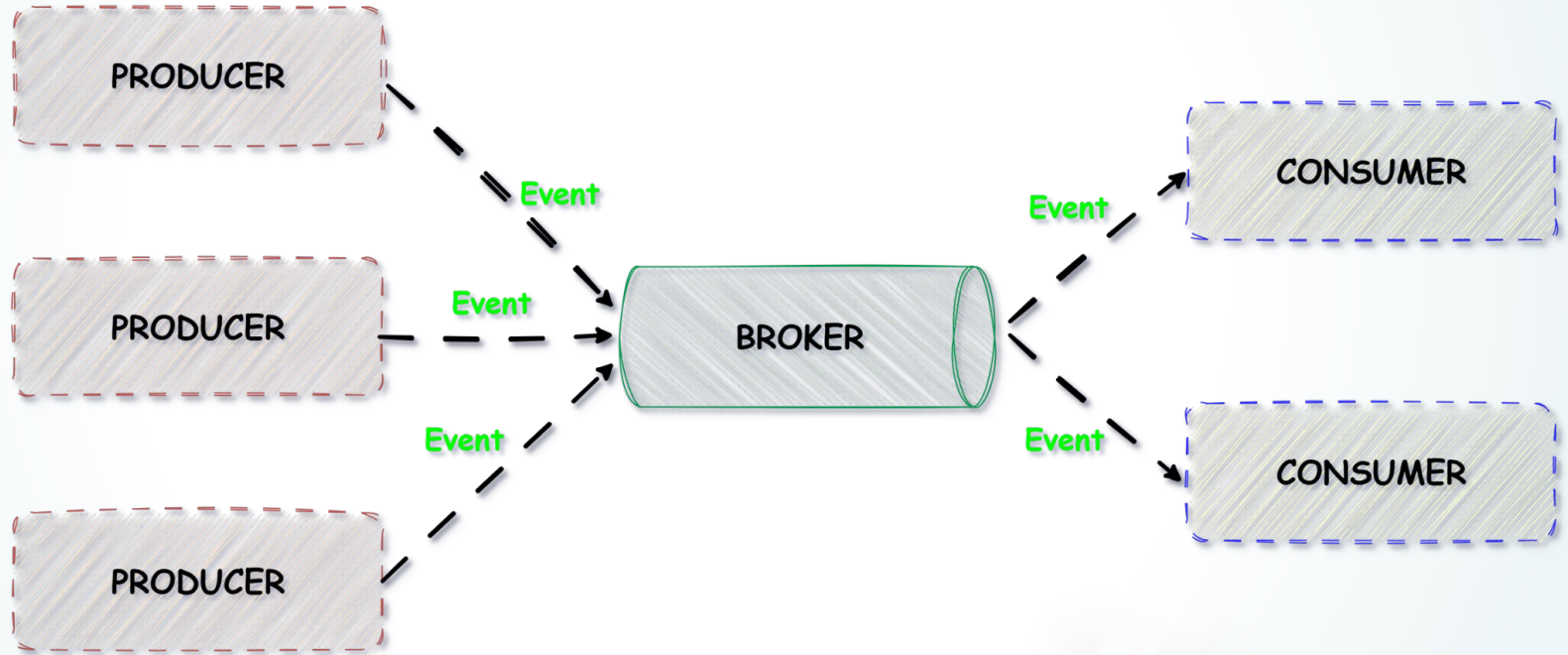
Middleware that receives, stores,
and routes events.
Examples: Kafka, RabbitMQ, NATS.



CONSUMER

Subscribes to events and reacts
accordingly.
Example: Payment service processes
“OrderPlaced”.

Key Components



Other Components

 Dispatcher	 Aggregator	 Listener
In some EDA implementations, a dispatcher is used to route events to the appropriate event handlers.	Several related events are combined by an aggregator to create a single, more significant event.	A component that actively monitors the event bus for events and reacts to them.
This part controls how events go through the system, making sure they are routed to the appropriate locations for processing.	This reduces complexity by simplifying the management of numerous individual events, making it easier for subscribers to process relevant information.	Listeners are often tailored to specific event types, ensuring that they respond promptly to changes that are relevant to their function.

Event Brokers

An Event Broker is a middleware component in Event-Driven Architecture that receives events from producers, stores and routes them, and delivers them to interested consumers. It acts as the central nervous system of the EDA ecosystem, ensuring that events move reliably, securely, and efficiently between decoupled services.



EDA Design Pattern

Event Notification

□ A lightweight messaging style where an event signals that something happened without including full state details

Event Sourcing

□ A pattern where the state of an application is derived by storing and replaying a complete history of events rather than persisting the current state directly.

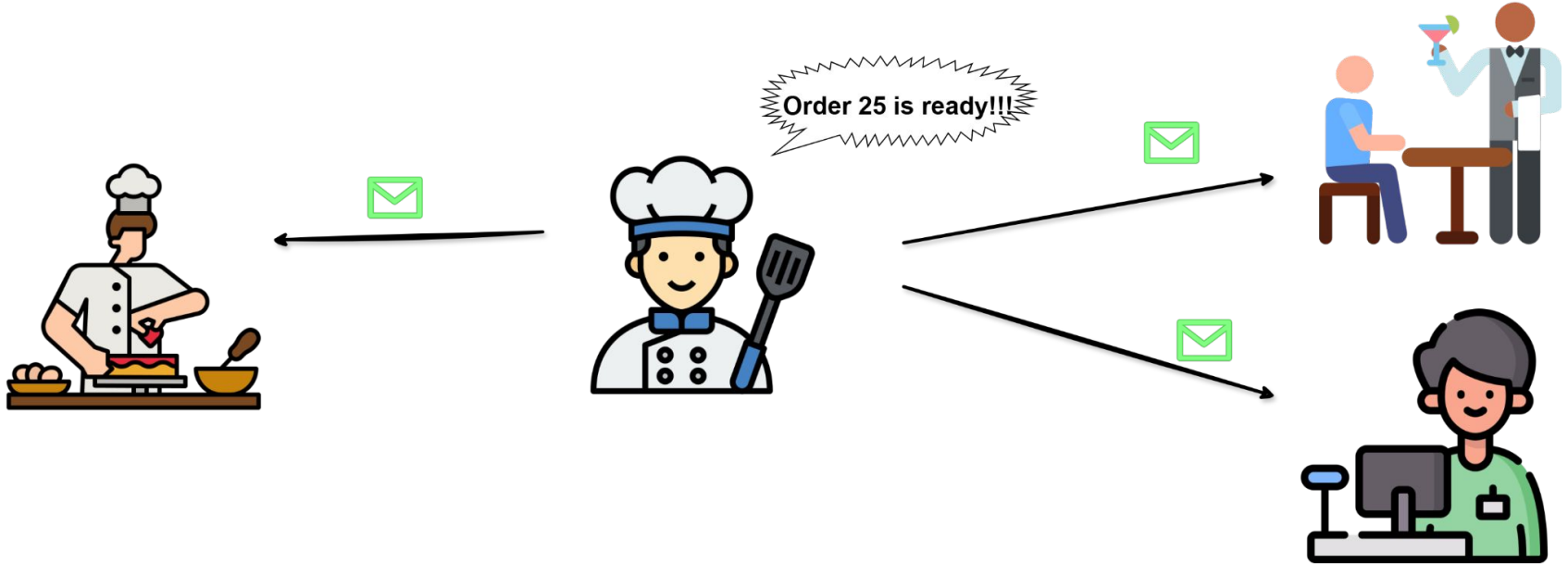
Event Carried State Transfer

□ A messaging pattern where events include the full state needed for consumers to process them without additional lookups

Command Query Responsibility Segregation (CQRS)

□ A pattern where commands trigger events to update write models, and events feed read models for optimized queries.

Event Notification



Event Notification

- ☀️ *Sends a simple signal that something happened, without full data.*
- ☀️ *Allows multiple listeners to react independently to the same event.*
- ☀️ *Reduces message size and processing overhead.*

When to use:

- ☀️ *Use it when consumers only need to know that an event occurred and can fetch more details if necessary.*



Event Carried State Transfer



Principal

Parent Invitation Letter

From: Mr. Principal
To: Lisa's parents
Date: Saturday, 14th September
Location: Principal's Room
Reason: Naughty student
Contact: Mr. John +1 23 456 789



Event Notification

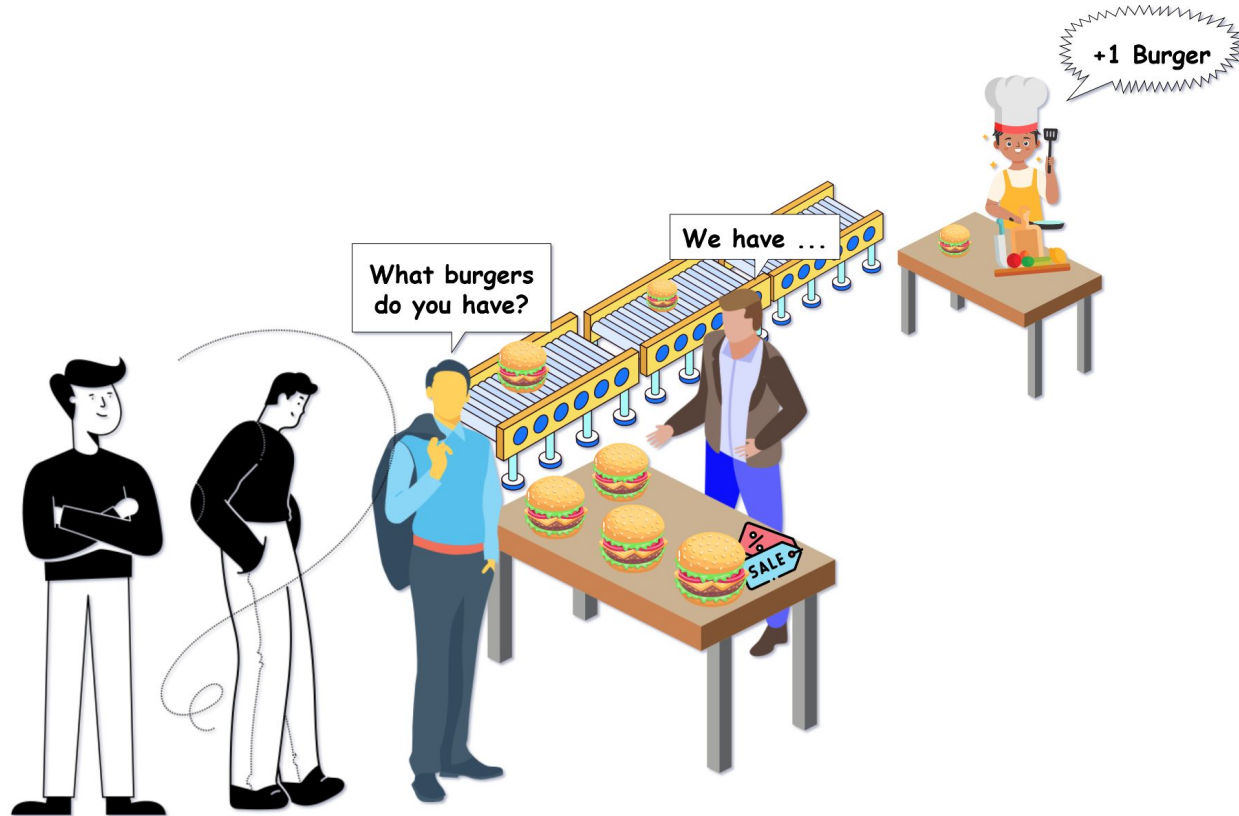
- ☀ *Each event contains all the data needed for consumers to process it without extra lookups.*
- ☀ *Reduces dependency on the original source system after the event is published.*
- ☀ *Useful for building read models or enabling offline processing.*

When to use:

Use it when consumers need complete information in one go to act immediately without fetching more data.



Command Query Responsibility Segregation (CQRS)



Event Sourcing

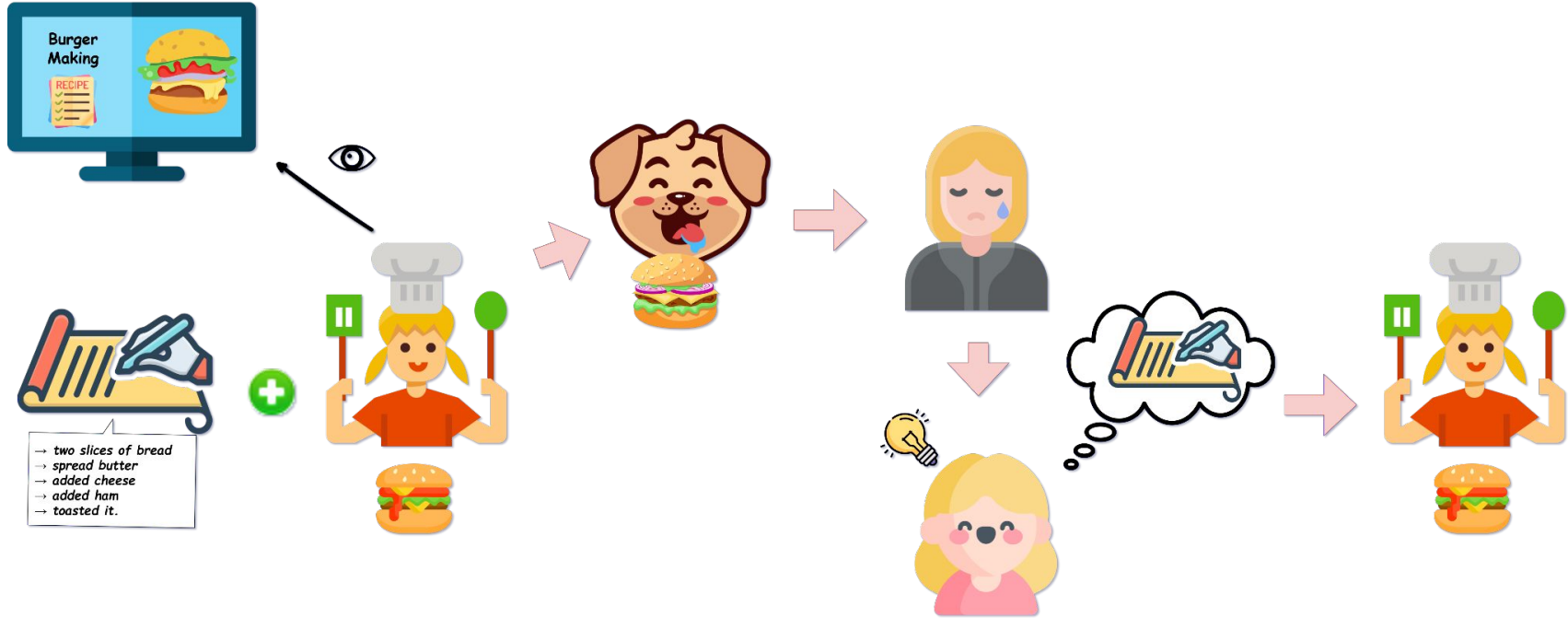
- ✨ *Separates reads (queries) and writes (commands) into different models for efficiency.*
- ✨ *Uses events from the write side to update the read side and notify other services.*
- ✨ *Improves scalability and flexibility by allowing each side to evolve independently.*

When to use:

Use it when your system needs high performance for both reading and writing and must keep read models updated through events in real time.



Event Sourcing



Event Notification

- ☀ Stores every state change as an immutable sequence of events.
- ☀ Current state is rebuilt by replaying all past events.
- ☀ Provides a complete audit trail and allows reconstructing or altering state from history.

When to use:

Use it when you need a full history of changes and the ability to rebuild or debug state at any point in time.



Trade-offs

Trade-off



Eventual Consistency

Data may take time to sync across services.

Use **idempotent consumers** to avoid duplicate processing.



Debugging Complexity

Tracing async flows can be harder than synchronous calls.

Implement **distributed tracing** (e.g., OpenTelemetry, Jaeger).



Operational Overhead

Brokers need monitoring, scaling, and management.

Monitor brokers with proper **metrics** and **alerts**.



Message Duplication / Ordering

Extra logic needed to handle duplicates or preserve order.

Apply **transactional outbox** or **exactly-once semantics** where needed.

When not use EDA?



Simple, CRUD-Heavy Applications

- EDA adds unnecessary complexity for small apps with basic create/ read/ update/ delete flows.



Strong Consistency Requirements

- If your system cannot tolerate eventual consistency (e.g., financial ledger updates), EDA may not be a good fit without complex compensation logic.

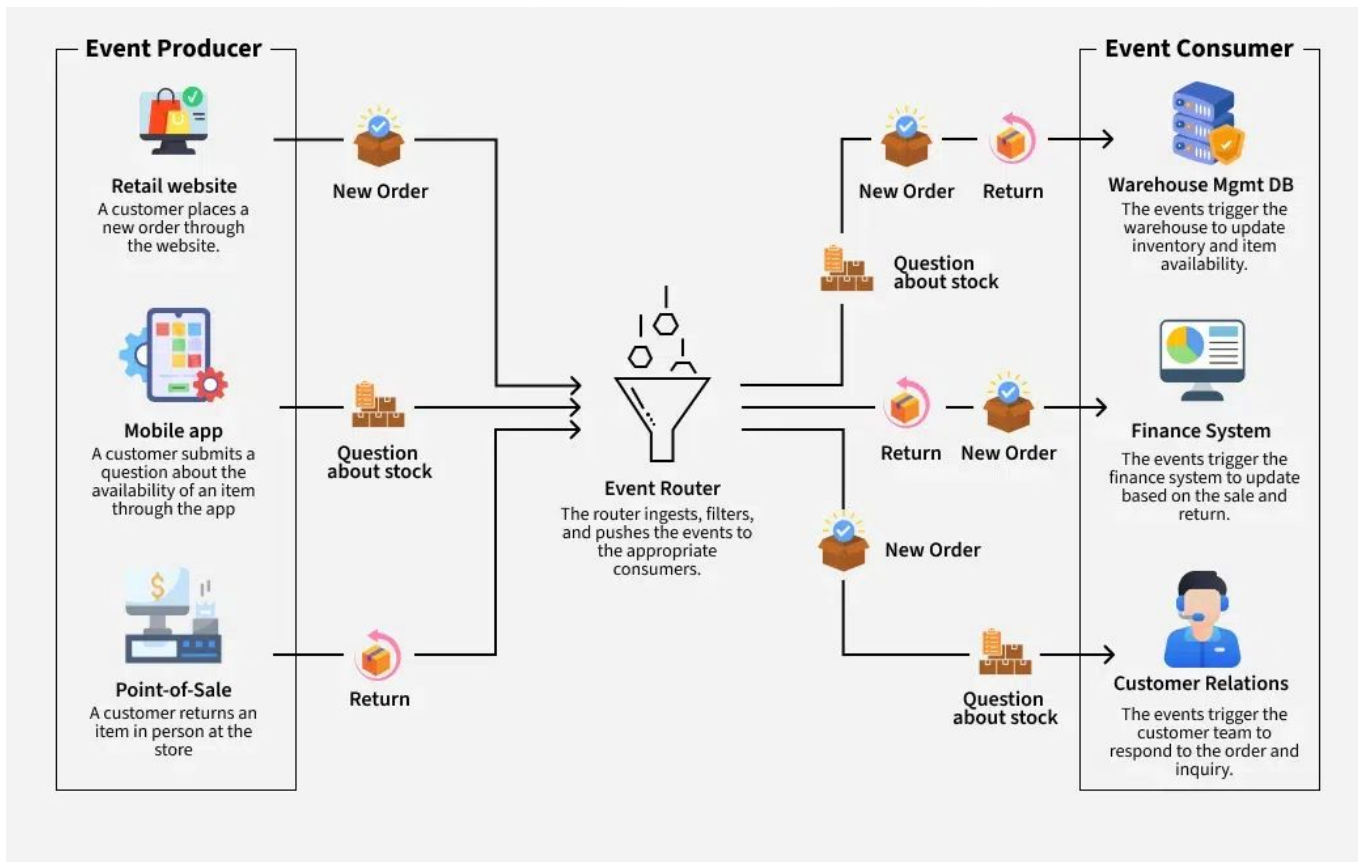


Limited Team Expertise & Resources

- EDA requires skills in async programming, distributed systems, and message broker management—teams lacking this may face steep learning curves.

Demo: Building an event-driven system

Demo: Architecture



Discussion and Q&A



Thank you!
